Deep Learning
DSAI 308

Prepared By
Dr. Khaled Mostafa ElSayed
kmelsayed@zewailcity.edu.eg

Instructors:

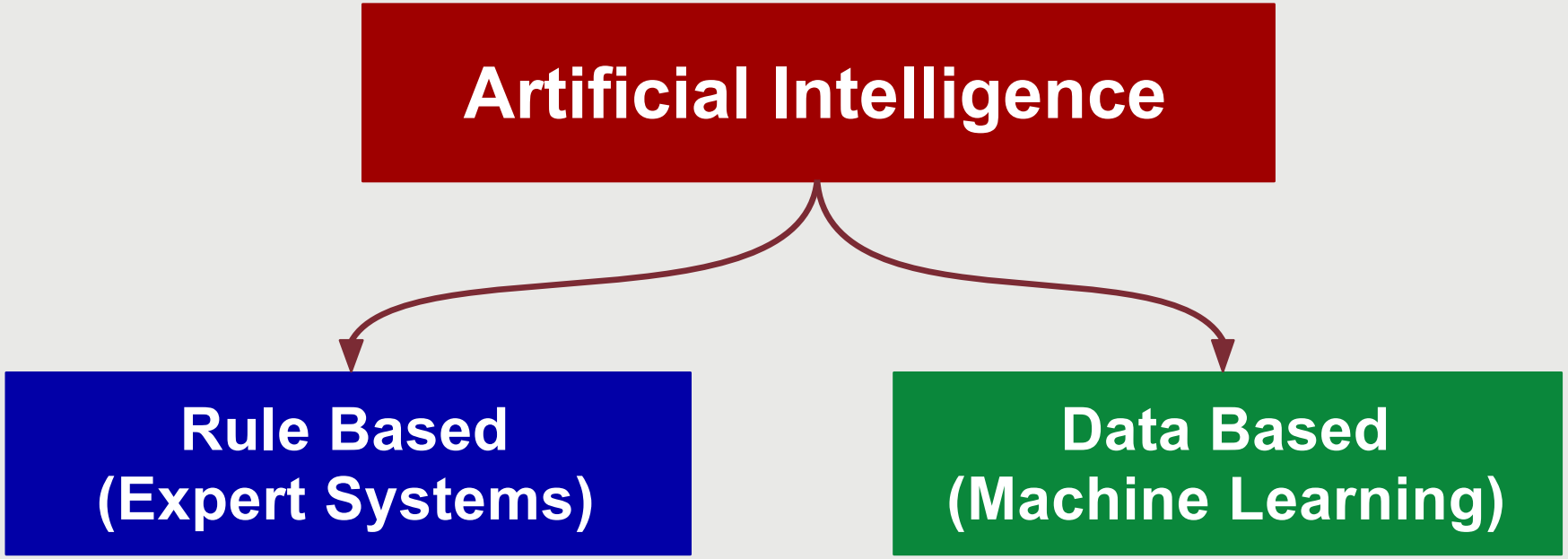
Dr. Khaled Mostafa ElSayed
kmelsayed@zewailcity.edu.eg
Dr. Mohamed Maher Ata
momaher@zewailcity.edu.eg



Agenda

- Quick revision of Machine Learning
- Deep Learning vs Machine Learning
- Quick revision of Neural Network
- Introduction to Convolutional Neural Network

[1] Revision of Machine Learning


Artificial Intelligence
Rule Based
(Expert Systems)
Data Based
(Machine Learning)

What is Machine Learning?

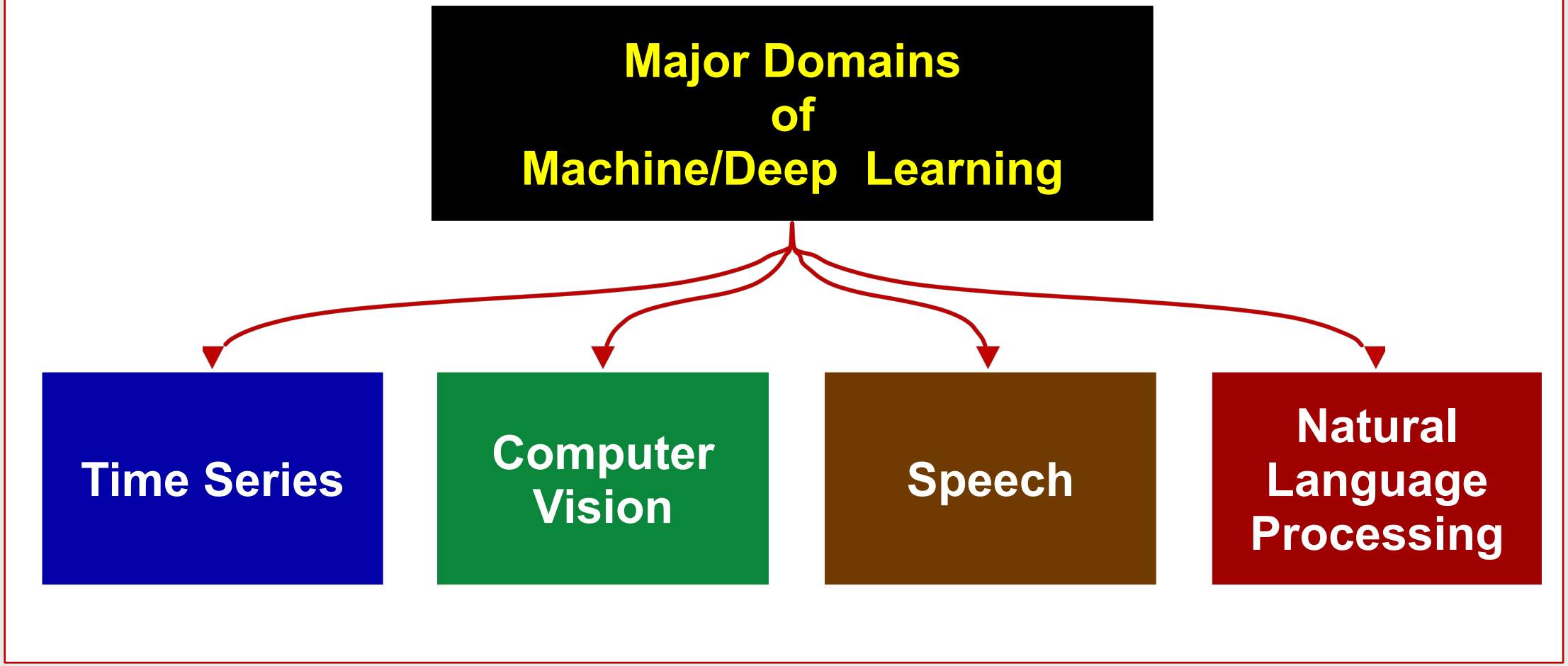
Machine Learning:

- Designing algorithms that can learn patterns from data (and exploit them)
- Approach: human supplies training examples, the machine learns
- Example: Show the machine a bunch of spam and legitimate emails and let it learn to predict if a new email is spam or not

Machine Learning in the real-world

Broadly applicable in many domains (e.g., finance, robotics, bioinformatics, vision, natural language, etc.). Some applications:

- Spam filtering
- Machine Translation (e.g., Google Translate)
- Speech/handwriting recognition
- Object detection/recognition
- Weather prediction
- Classifying DNA sequences
- Stock market analysis
- Automatic vehicle navigation
- Search engines (e.g., Google)
- Credit-card fraud detection
- Webpage clustering (e.g., Google News)
- .. and many more


Major Domains
of
Machine/Deep Learning
Time Series
Computer Vision
Speech
Natural Language Processing

Programming Languages, Platforms, and Libraries

Popular Languages for ML/DL

- Python
- R
- Matlab

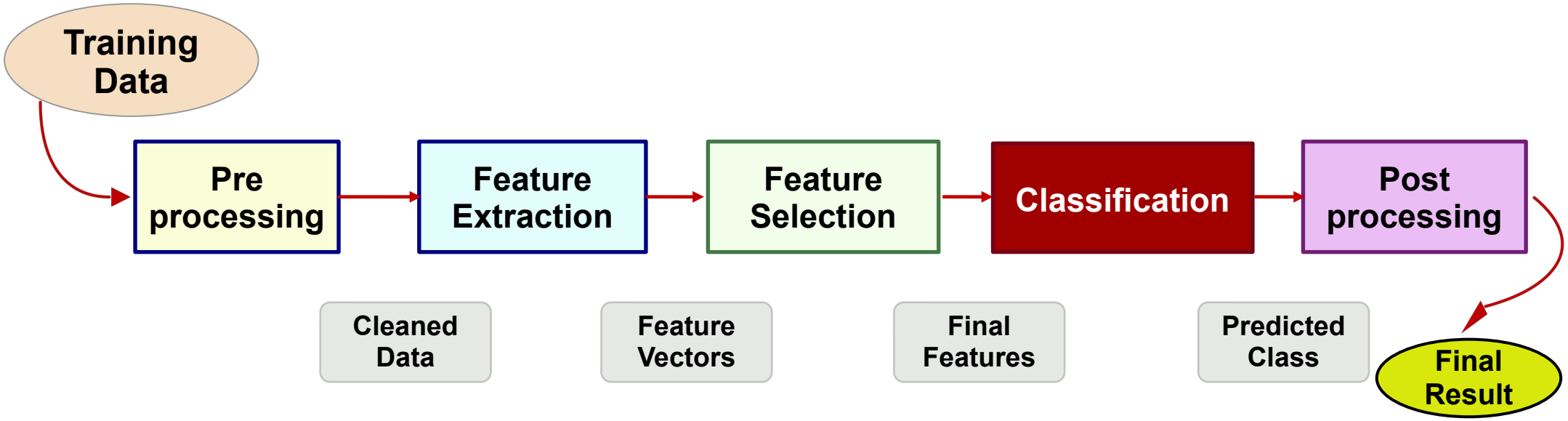
Popular Platforms for ML/DL

- Tensor Flow
- Pytorch

Popular Libraries (for python)

- SciKit Learn
- SciPy
- NLTK
- Spacy
- Gensim
- OpenCV

The Complete ML Classification Process



Pre processing

Preprocessing is *Problem dependent and data dependent*

It may include:

- Gap filling
- Thinning
- Contour
- Binarization
- Noise Removal
- Color to Gray
- Opening touched points
- Resizing
- Normalization
- .....

Feature Extraction

Features Types are is ***Problem dependent***

Features may include:

- Center of Gravity location,
- Aspect Ratio,
- Length and Width,
- Number of intersections,
- Moments,
- Geometrical Features ,
- Colors,
- Transformations
- ……

Feature Invariants

Features Invariants are is *Problem dependent*

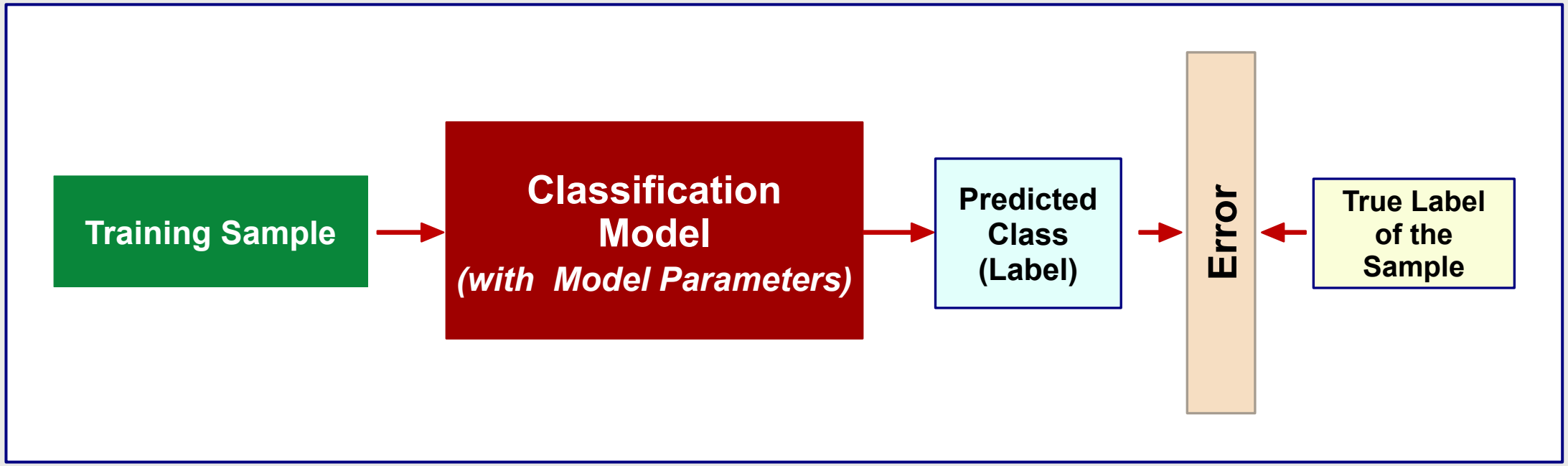
- **Rotation Invariant**
- **Scaling Invariant**
- **Translation invariant**
- **Mirroring Invariant**

Classification

Examples of Commonly used classifiers:

- Linear Regression
- Logistic Regression
- Neural Network
- Support Vector Machine
- Hidden Markov Model
- Decision Trees

How Training Takes Place to learn the model ?



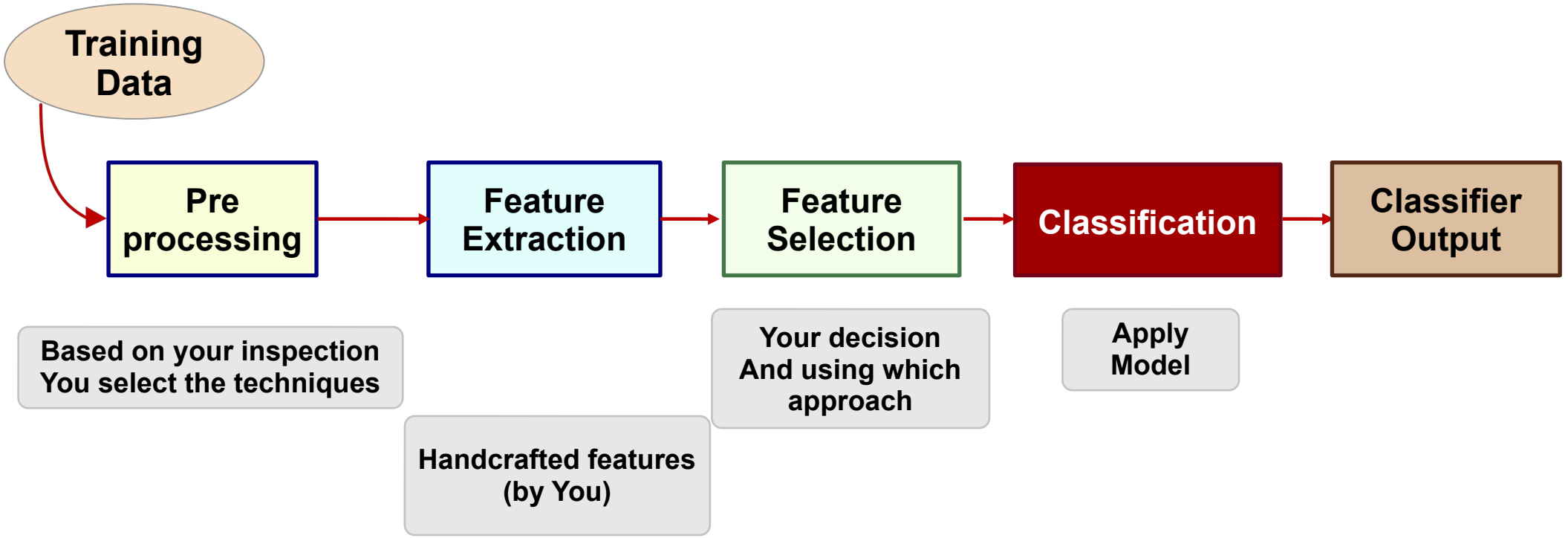
Try to update model parameters (Iteratively) to minimize error (Loss Function)

[2] Machine Learning

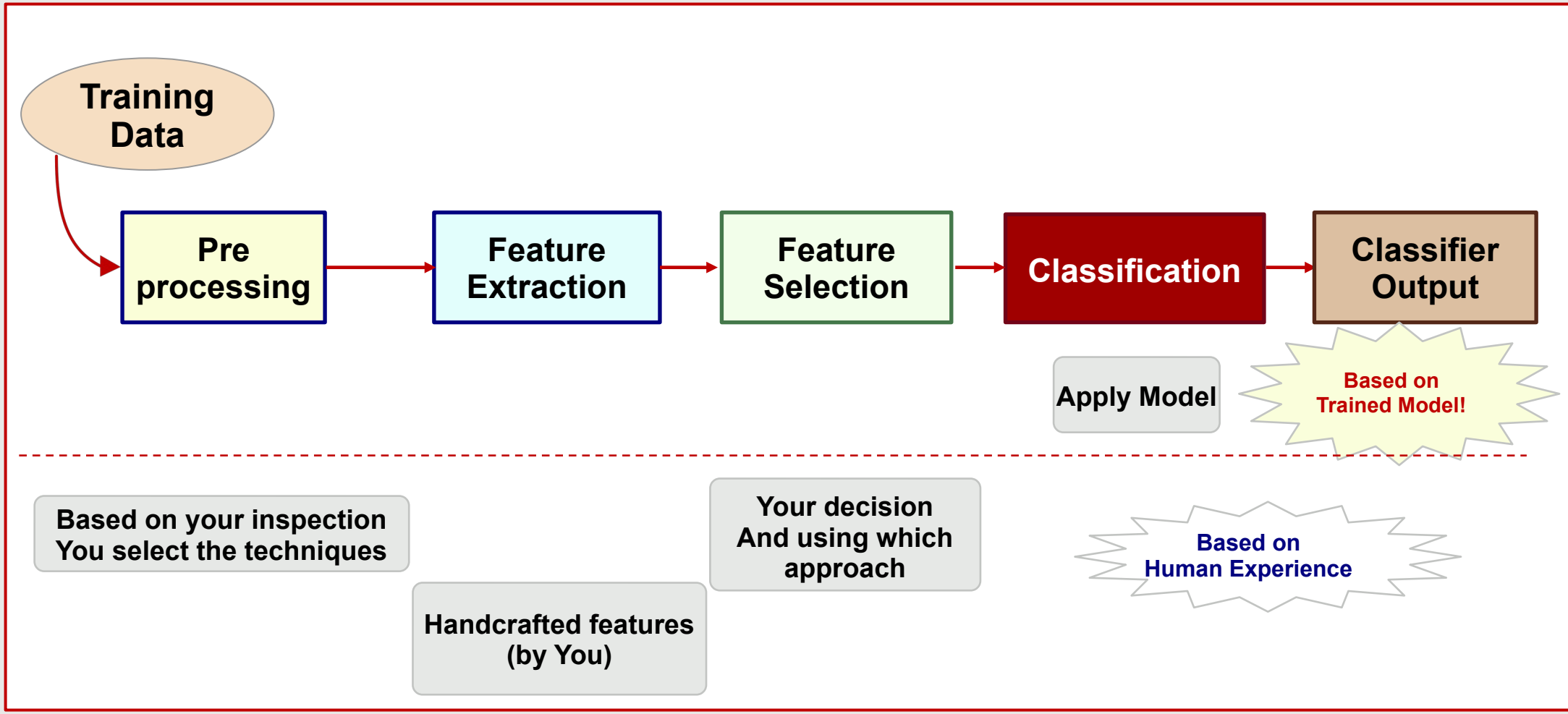
vs

Deep Learning

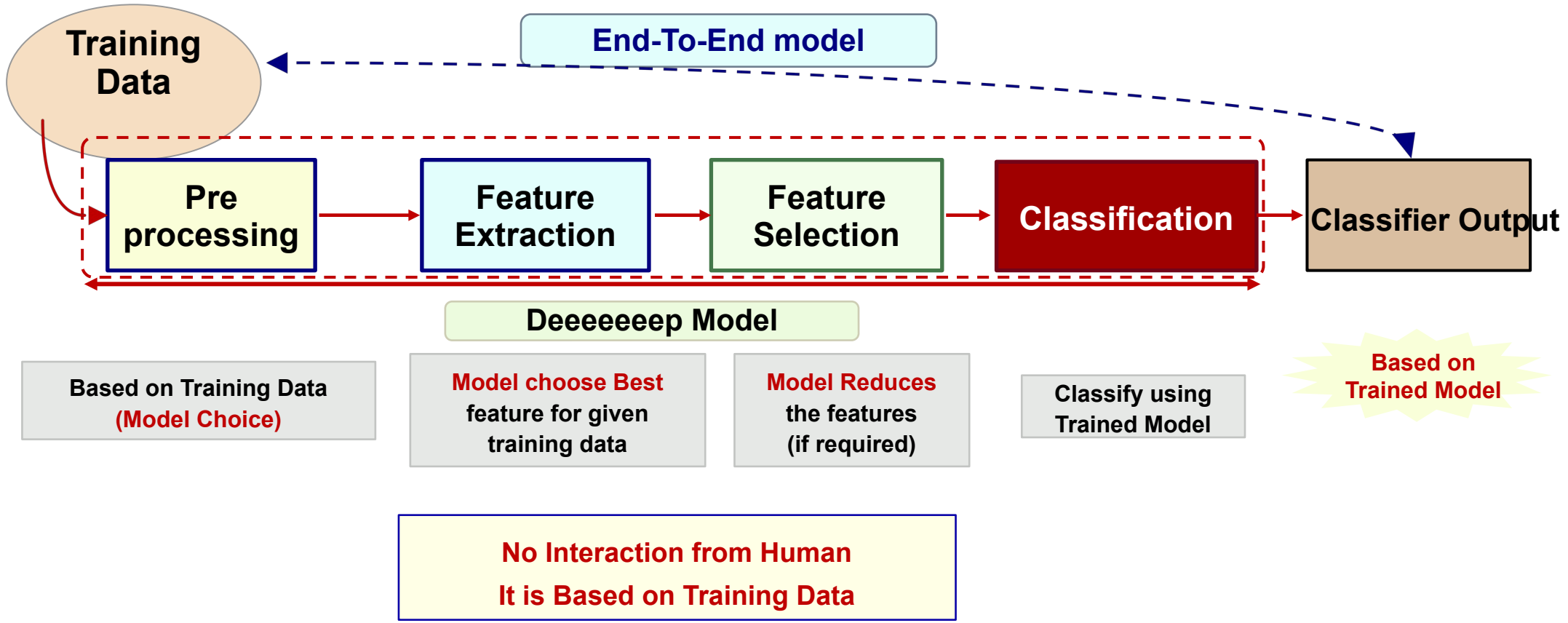
The Complete ML Classification Process



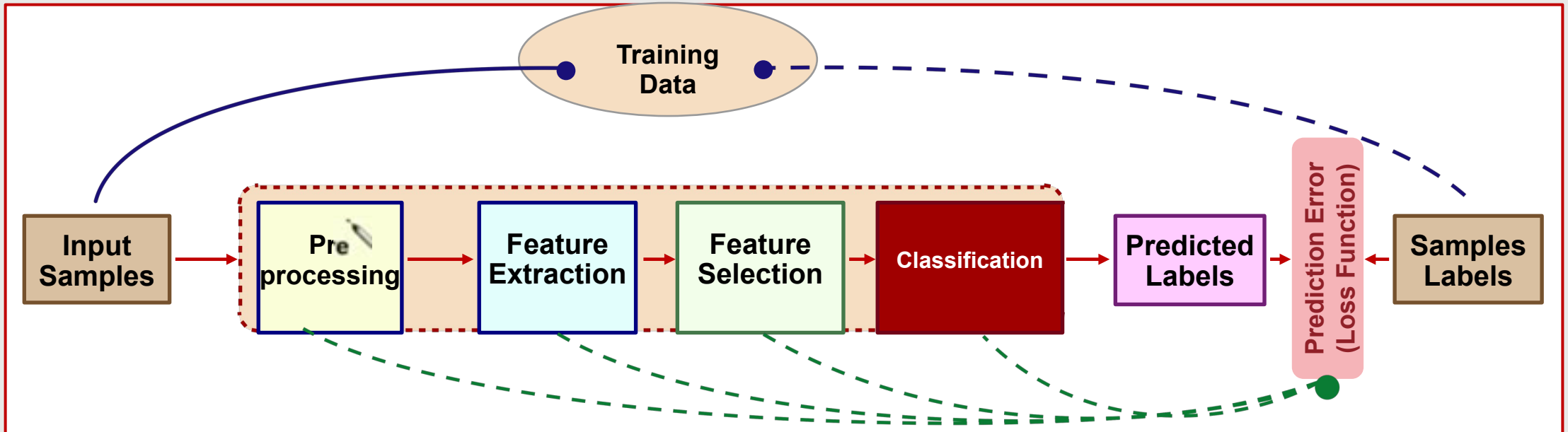
The Complete ML Classification Process



The Complete Deep Learning Model

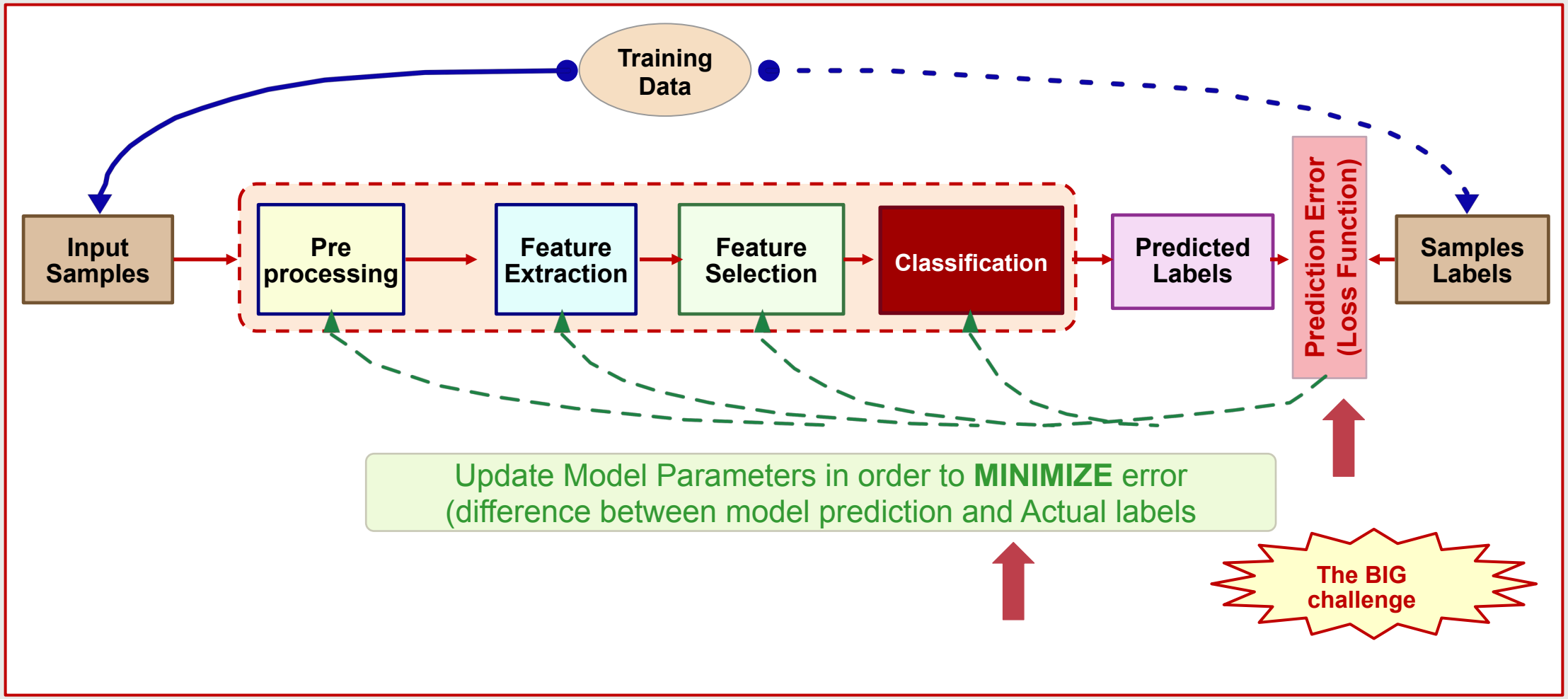


The Complete Deep Learning Model



Update Model Parameters in order to **MINIMIZE** error
(difference between model prediction and Actual labels

The Complete Deep Learning Model



[3] Quick revision of Neural Netwok (NN)

Visualization of Single Perceptron

$$\underline{\mathbf{X}} = \begin{pmatrix} x_1 \\ x_2 \\ \dots \\ \dots \\ x_n \end{pmatrix} \Rightarrow \underline{\mathbf{X}} = \begin{pmatrix} 1 \\ x_1 \\ x_2 \\ \dots \\ \dots \\ x_n \end{pmatrix}$$

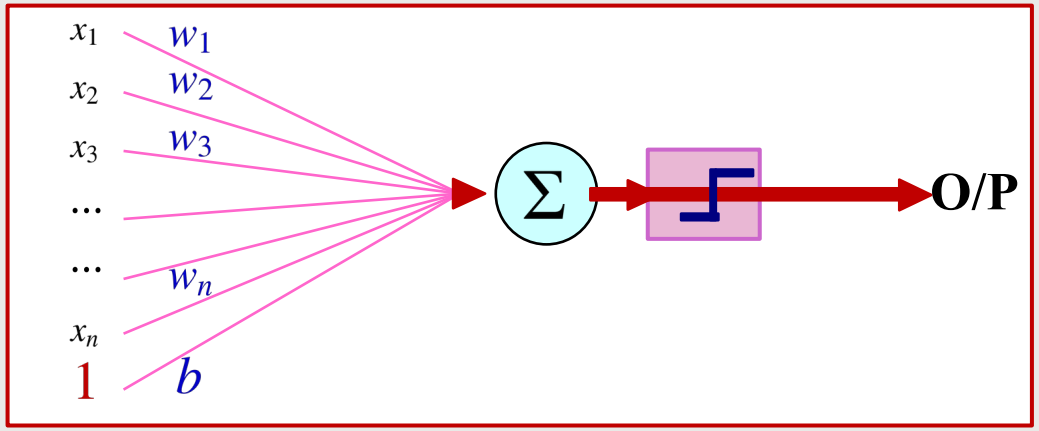
Ugmented Vector

$$\underline{\mathbf{W}} = \begin{pmatrix} b \\ w_1 \\ w_2 \\ \dots \\ \dots \\ w_n \end{pmatrix}$$

Model Weights

$$d(\mathbf{X}) = \underline{\mathbf{W}}^T \underline{\mathbf{X}} = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b * 1 \Rightarrow$$

$> 0 \Rightarrow$ Class I
$< 0 \Rightarrow$ Class II
$= 0 \Rightarrow$ Undefined Class



Binary Classification (2 Classes)

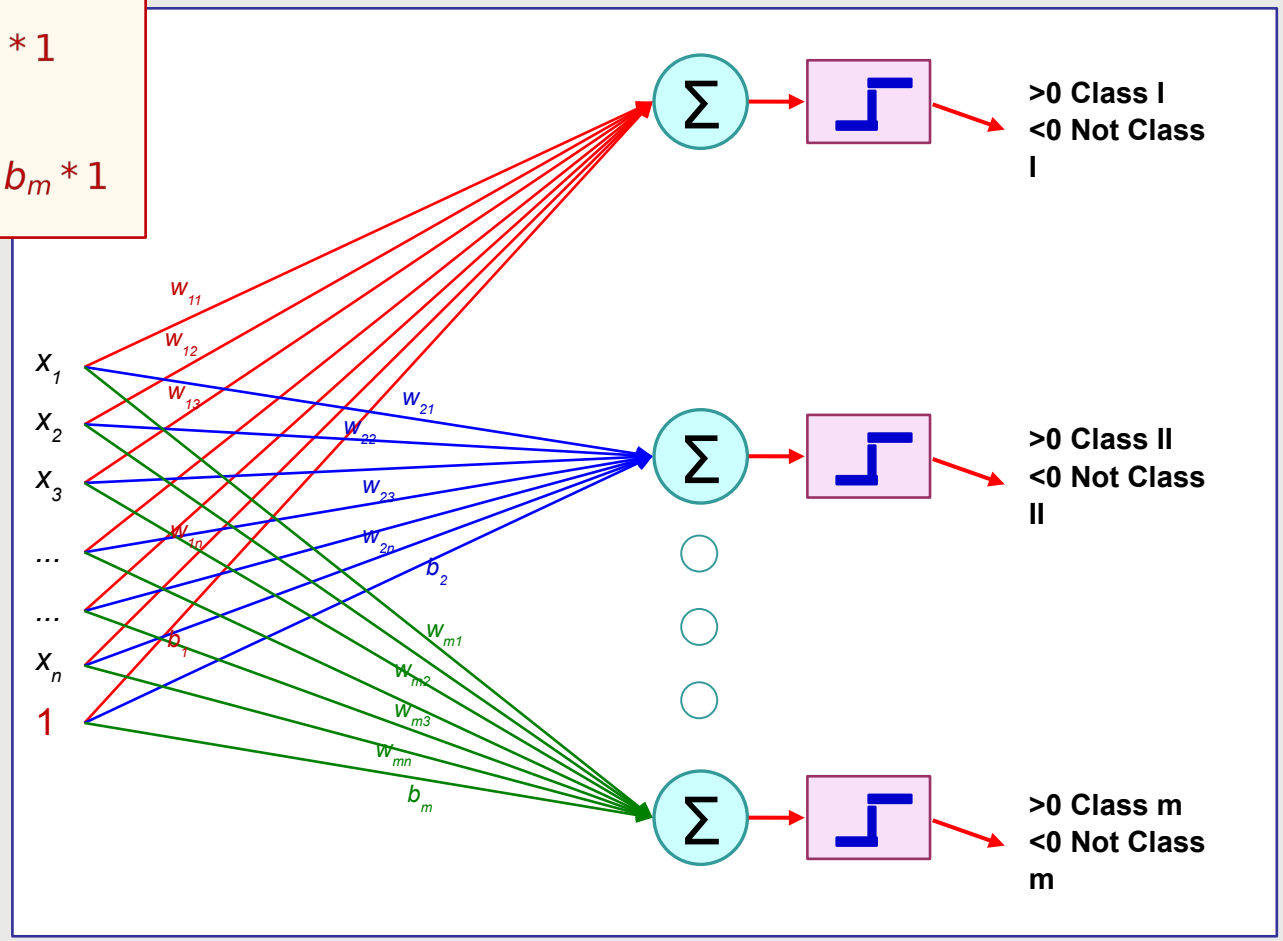
Visualization of Single Layer Perceptron

$$\boldsymbol{d}_1(\boldsymbol{X}) = \underline{W_1}^T \boldsymbol{X} = w_{11}x_1 + w_{12}x_2 + \ldots + w_{1n}x_n + b_1 * 1$$

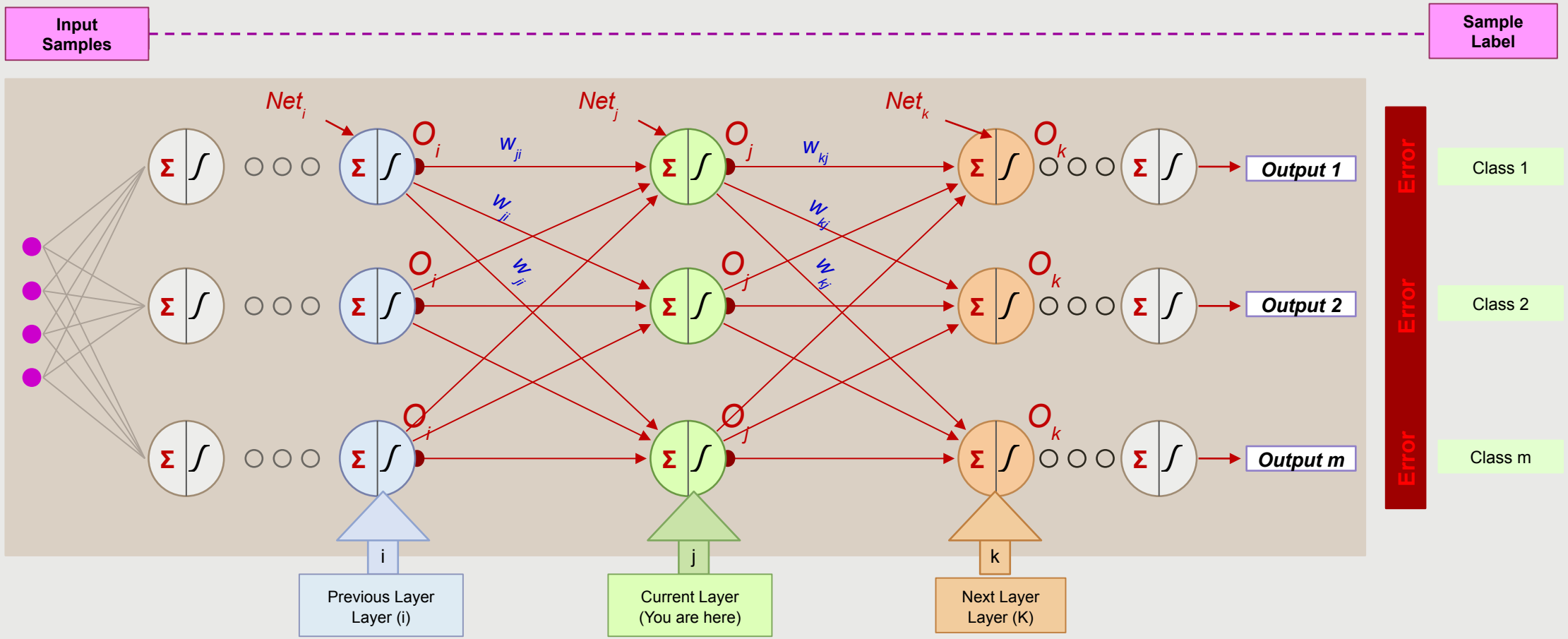
$$\boldsymbol{d}_2(\boldsymbol{X}) = \underline{W_2}^T \boldsymbol{X} = w_{21}x_1 + w_{22}x_2 + \ldots + w_{2n}x_n + b_2 * 1$$

$$\boldsymbol{d}_m(\boldsymbol{X}) = \underline{W_m}^T \boldsymbol{X} = w_{m1}x_1 + w_{m2}x_2 + \ldots + w_{mn}x_n + b_m * 1$$

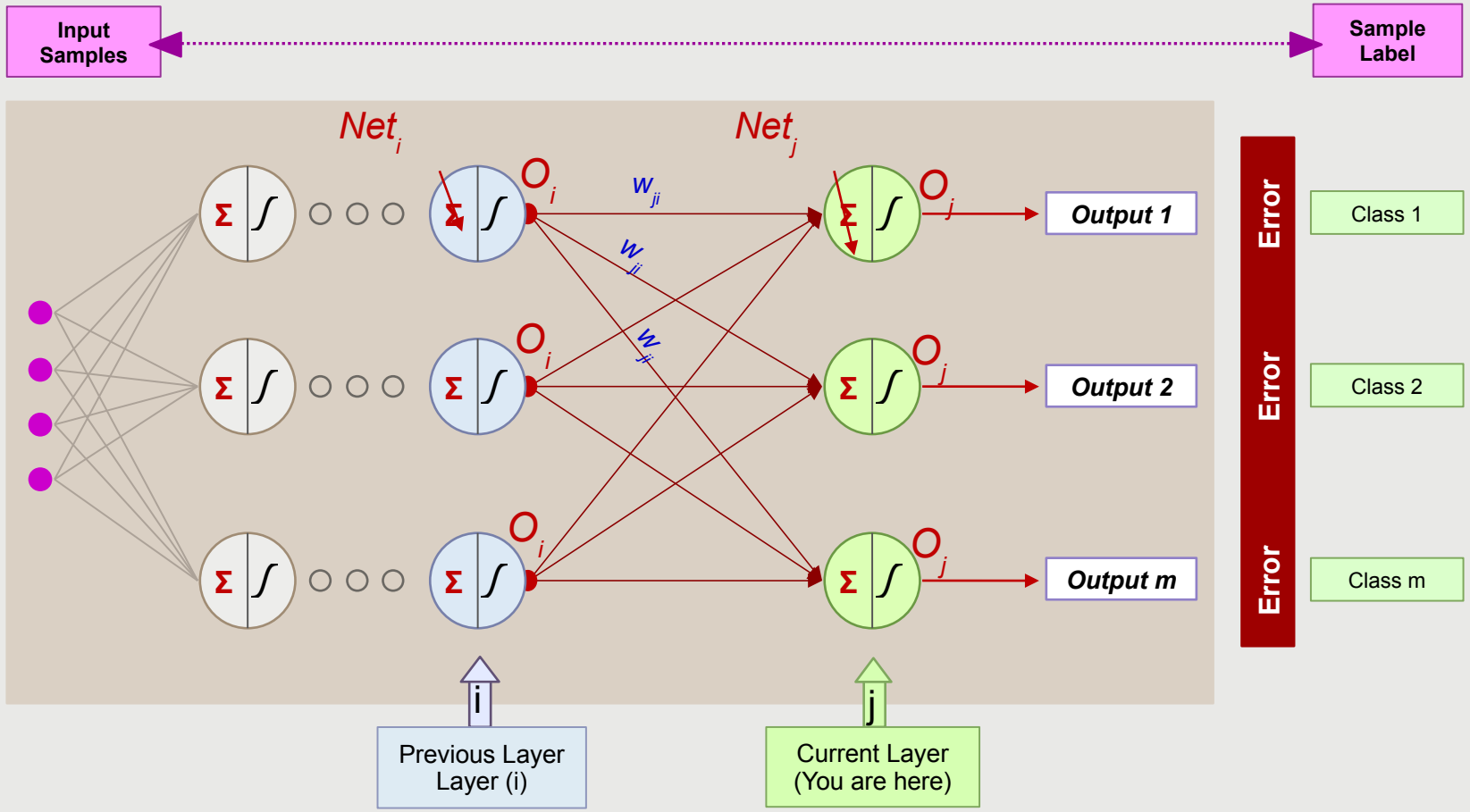
Multi-Class Classification



Neural Network Model



Neural Network Model



$$Net_j = \sum_i (w_{ji} O_i)$$

$$O_j = f(Net_j) = \frac{1}{1 + e^{-Net_j}}$$

$$E = \frac{1}{2} \sum_j (t_j - O_j)^2$$

Update the weights by moving in the negative direction of the Gradient of the Loss function with respect to model weights

$$Net_j = \sum_i (w_{ji}\, O_i) \quad \rightarrow \quad \frac{\partial\, Net_j}{\partial\, w_{ji}} = O_i$$

$$O_j = f(Net_j) = \frac{1}{1 + e^{-Net_j}}$$ (For Sigmoid Function)

$$E = \frac{1}{2}\sum_j (t_j - O_j)^2$$ (Error at Output layer ONLY)

$$\Delta w_{ji} = -\eta \frac{\partial\, E}{\partial\, w_{ji}} = -\eta \frac{\partial\, E}{\partial\, Net_j} \frac{\partial\, Net_j}{\partial\, w_{ji}}$$

$$\Delta w_{ji} = -\eta \frac{\partial\, E}{\partial\, Net_j}\, O_i$$ Let $$\delta_j = -\frac{\partial\, E}{\partial\, Net_j}$$

$$\Delta w_{ji} = \eta\, \delta_j\, O_i$$ then $$w_{ji}^{new} = w_{ji}^{old} + \Delta w_{ji}$$

[4] Introduction to

Deep Learning

and

CNN

Introduction to

Convolutional Neural Network (CNN)

Some of Slides are picked from
UVA Deep Learning Course EFSTRATIOS GAVVES

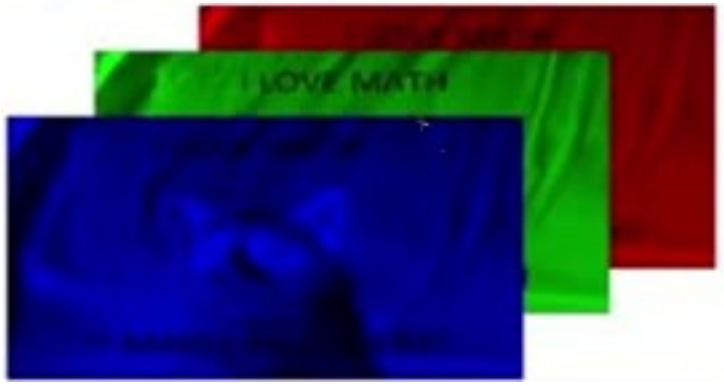
Spatial Structure of Image

- Images are 2-D
  - k-D if you also count the extra channels
  - RGB, hyperspectral, etc.



- What does a 2-D input really mean?
  - Neighboring variables are locally correlated





Apply filter on 2-D gray image (K=1 Channel)

e.g. Sobel 2-D filter





Where to Get Filters for Recognition

There are Several, handcrafted filters in computer vision

For example:
Canny, Sobel, Gaussian blur, smoothing, low- level segmentation, morphological filters, Gabor filters,...

- **Are they optimal for recognition?**
- **Can we learn them from our data?**

Filter Parameters

$$\begin{bmatrix} \theta_1 & \theta_2 & \theta_3 \\ \theta_4 & \theta_5 & \theta_6 \\ \theta_7 & \theta_8 & \theta_9 \end{bmatrix}$$

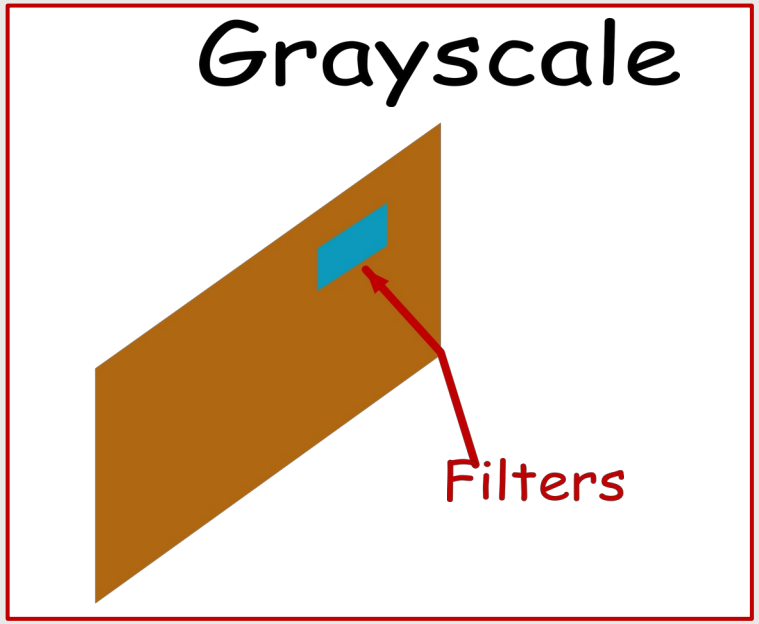
Where to Get Filters for Recognition

If images are 2-D,
parameters should also be organized in 2-D

That way they can learn the local correlations between input variables

That way they can "exploit" the spatial nature of images

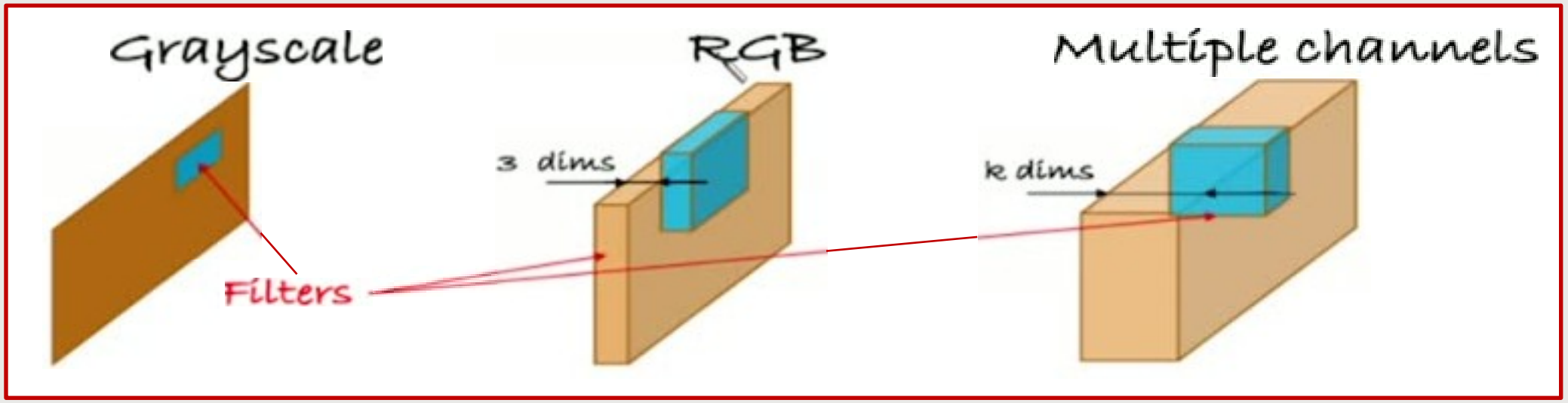
2-D Image



Where to Get Filters for Recognition

Similarly, if images are k-D, parameters should also be k-D

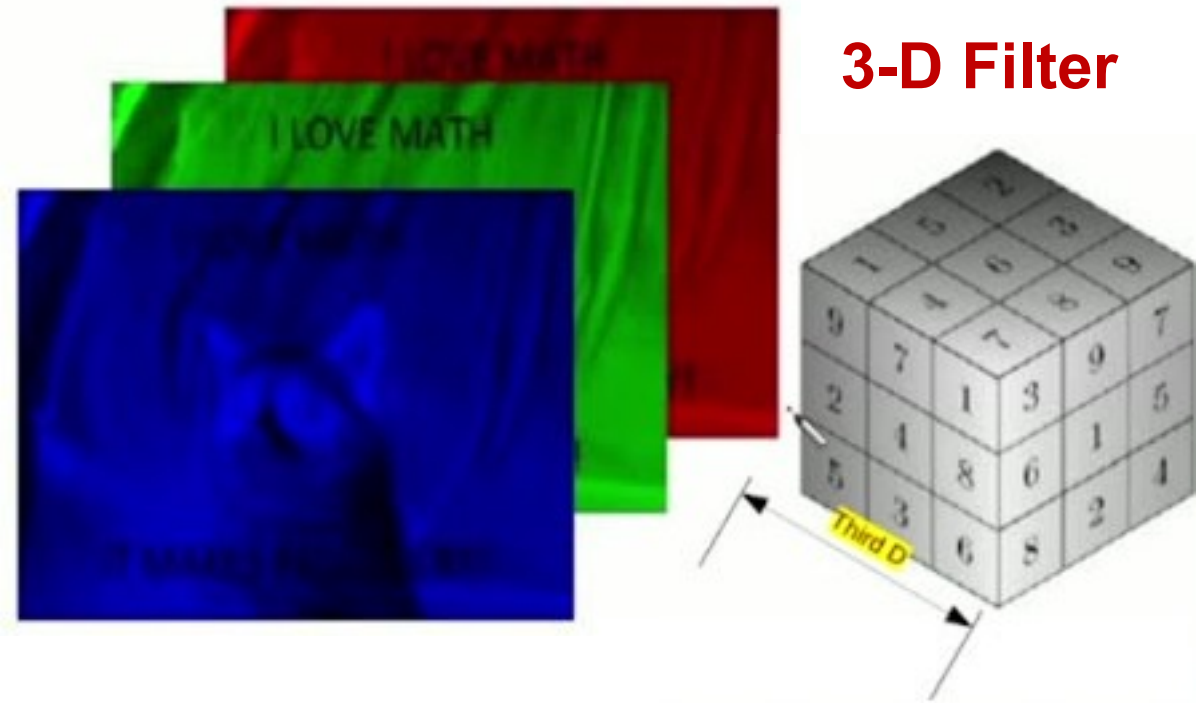
For K=3 (RGB), use K-D filter (third dim >> K=3)
For K=9 (Satellite images) use 9-D filter (3 rd dim = 9)
...etc



What would a k-D filter looks like?

e.g. Sobel 2-D filter





How to Apply Filter on image

Image

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

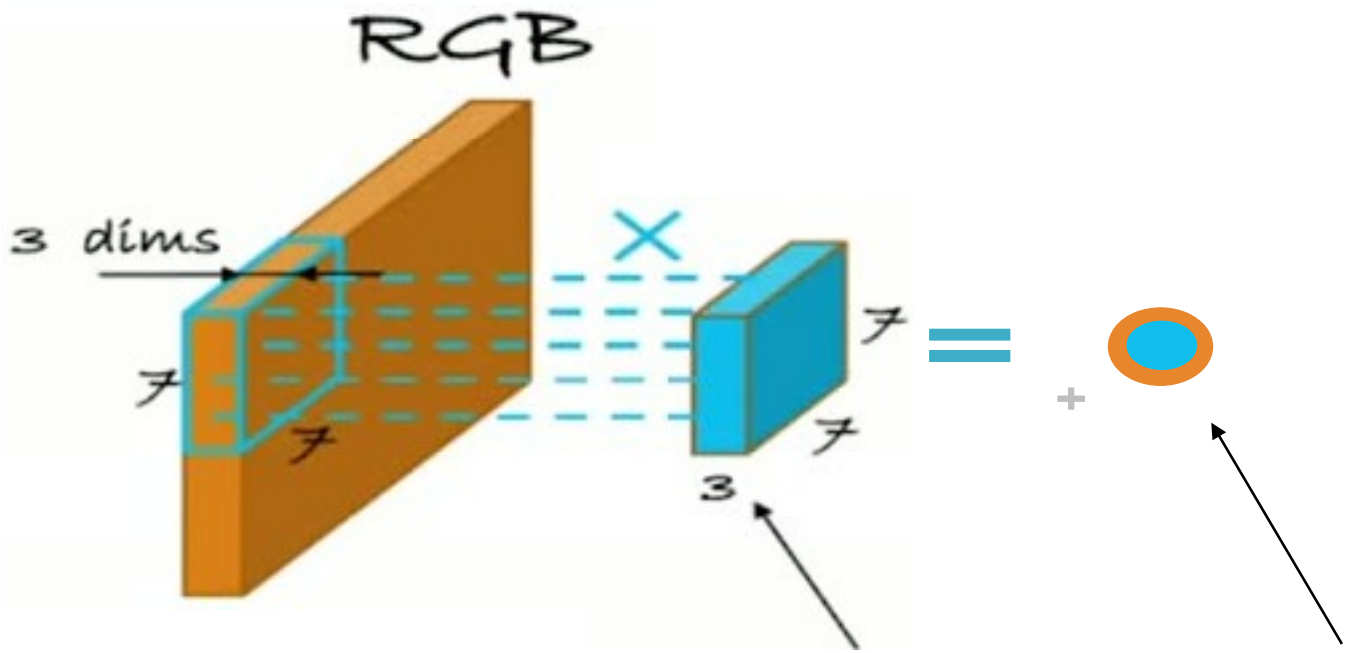
Filter

1	0	1
0	1	0
1	0	1



Apply Filter on Image is a Dot product between image values and filter values
(can be modeled by PERCEPTRON)

How many Parameters are in the RGB Image filter (K=3)

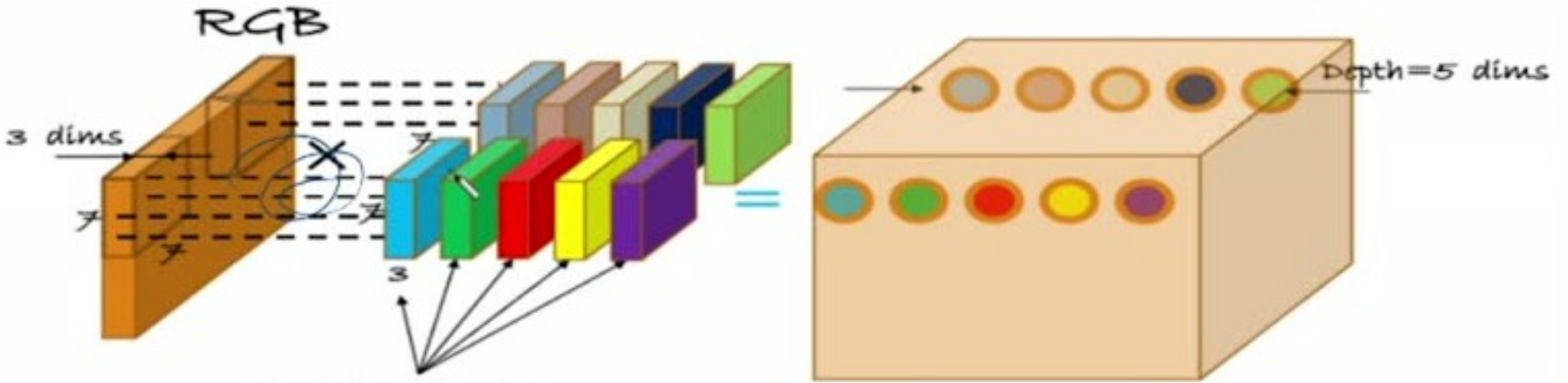


How many weights for this neuron?

$7 \cdot 7 \cdot 3 = 147$

How many filter can we apply on the image

We can Apply **multiple filters** on the **same location** (e.g. 5 different filters on the same location. For Example:
[one to **detect edges**, one to **detect lines**, one to **detect curves**, ...etc)



How many weights for these 5 neurons?

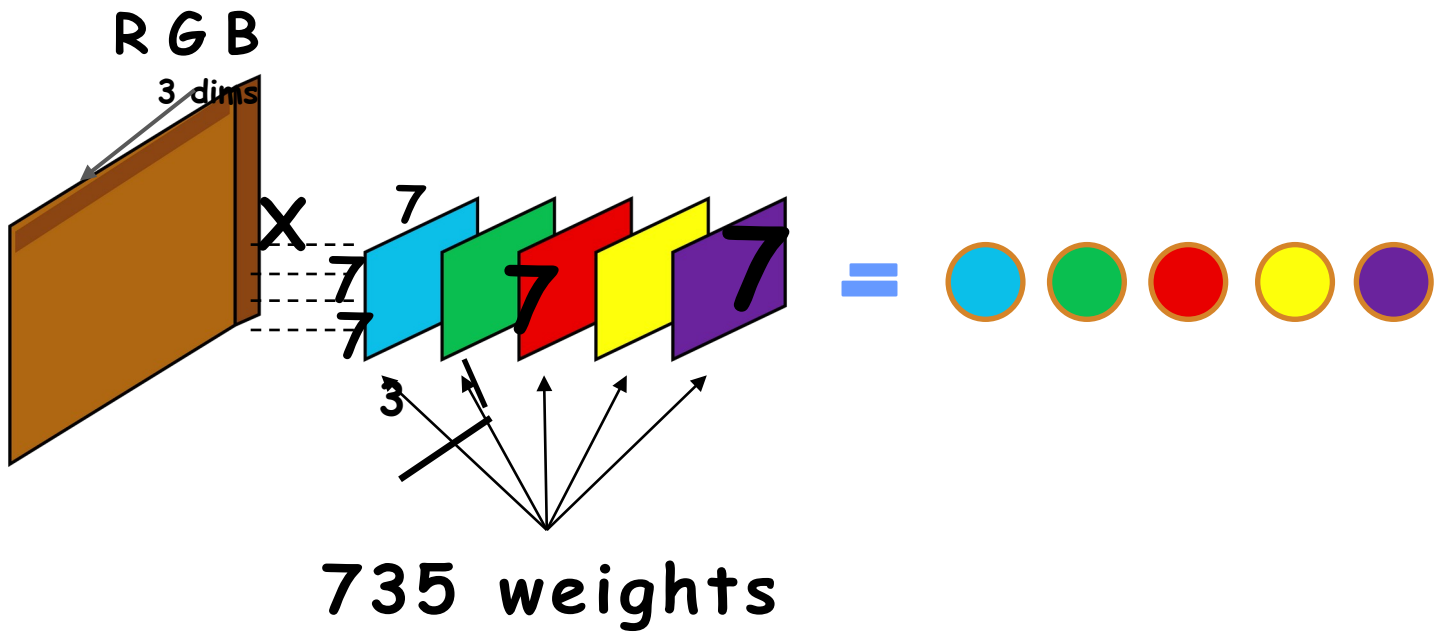
$5 \cdot 7 \cdot 7 \cdot 3 = 735$

Filters parameters

Number of parameters for different filters applied to same location in the image

= **Filter size (2-D)** * **Number of channels (K)** * **number of different filters**

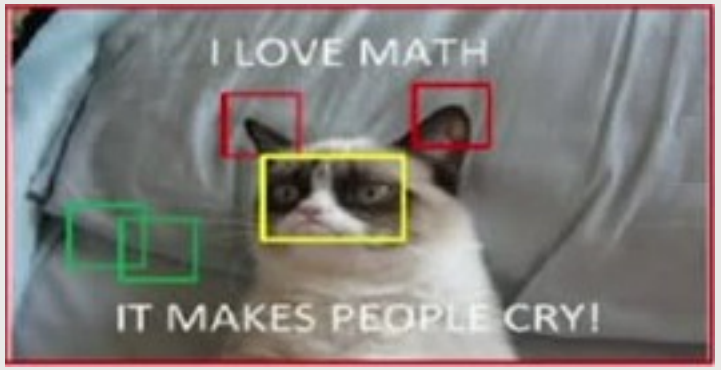
= **7*7** * **3** * **5** =735

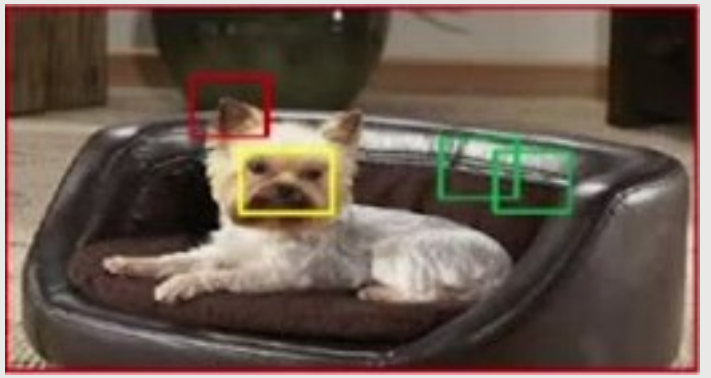


How many filters we should apply

Hypothesis
Natural images are stationary
Visual features are common for different parts of one or multiple image

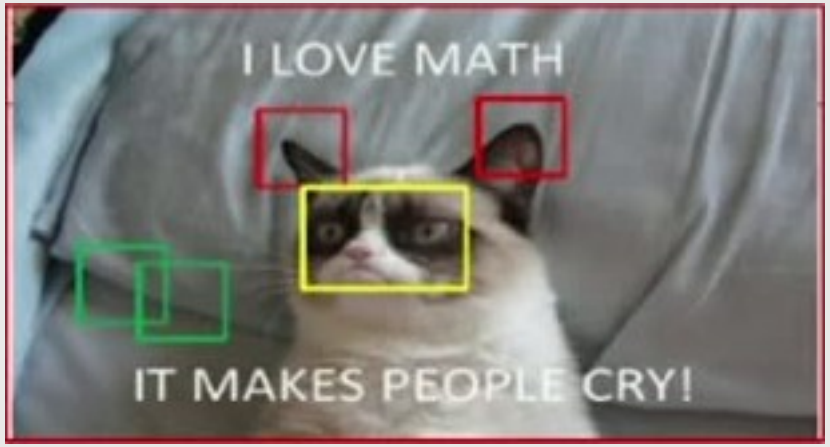
Conclusion
It is not required to use different filters for different spatial locations in the image (in other words, **apply same filter for ALL positions in the image**)

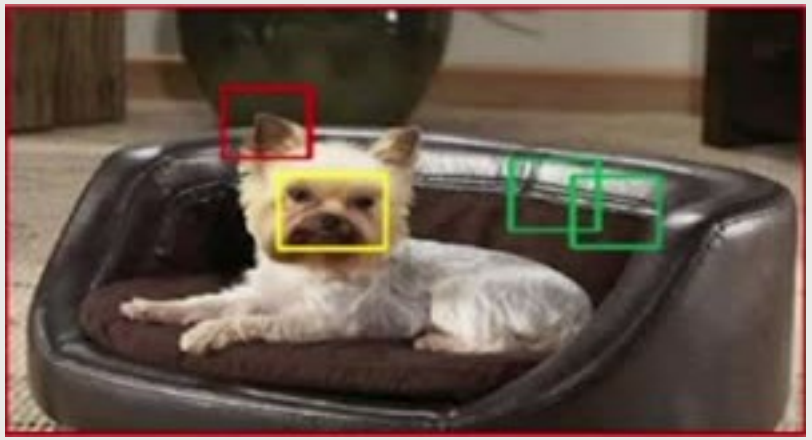




Apply Red filter to detect ears in the whole image (not necessarily in the middle of the image)
Apply filter to detect nose in the whole image
Apply Green filter to detect edges in the whole image

How many filters we should apply





Example: 30 different filters

Each filter can detect different information from the image

Apply those 30 filters on the image (each one should be applied on the whole locations on the image)



The Convolution Process

Original image



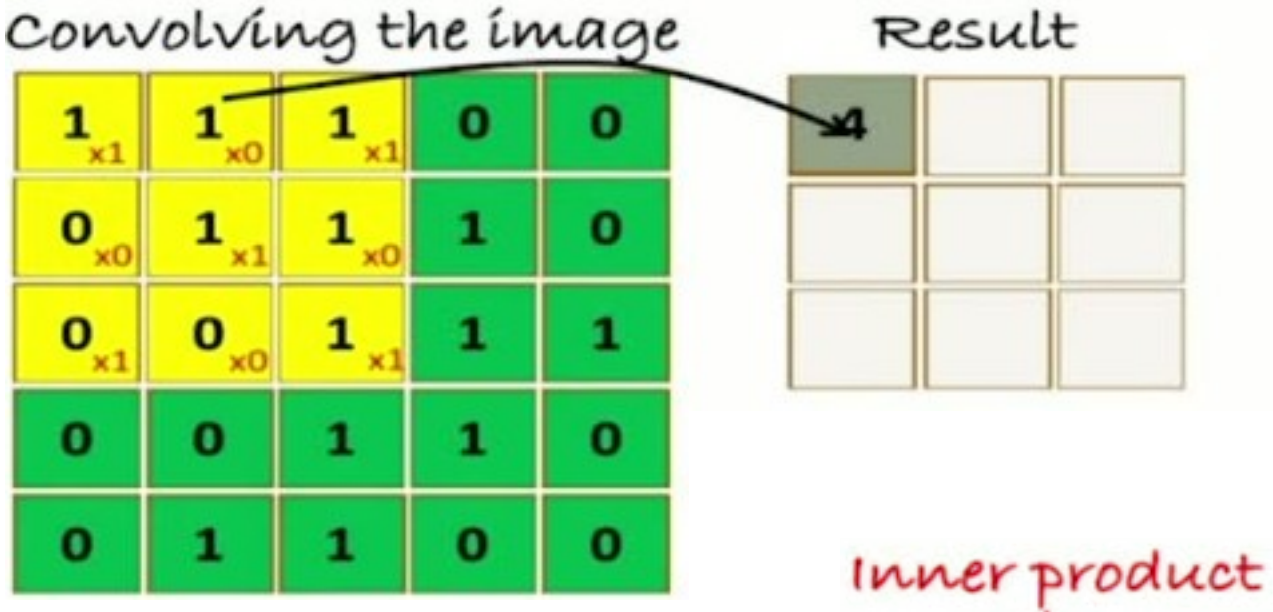
Convolutional filter 1

1	0	1
0	1	0
1	0	1

a=1, b=1

Image: I (x,y)

Filter: h (-a:a , -b:b) ***(odd dimension)***

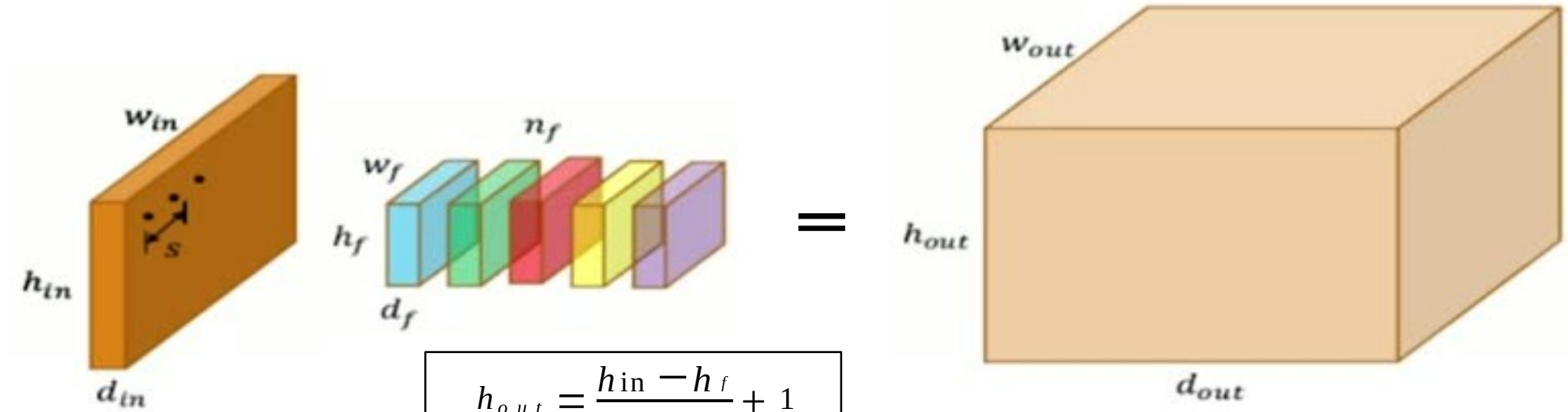


$$I(x,y) * h = \sum_{i=-a}^{a} \sum_{j=-b}^{b} I(x-i,\ y-j) \cdot h(i,j)$$

Scan the original image using the 3x3 filter

Move the filter ONE pixel (Right, Left, Up, Down) and Re-Calculate

Output dimensions of Convolution?



$$h_{out} = \frac{h_{in} - h_f}{s} + 1$$

$$w_{out} = \frac{w_{in} - w_f}{s} + 1$$

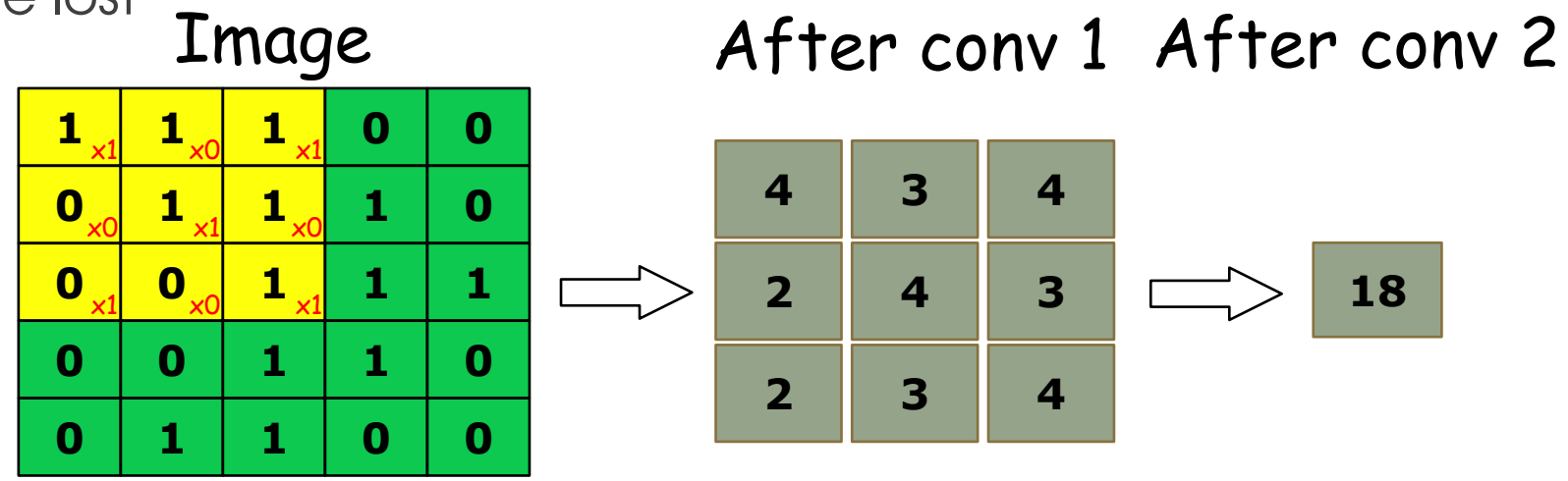
$$d_{out} = n_f$$

$$d_f = d_{in}$$

h	*hight*
w	*Width*
d	*Depth*
f	*Filter*
n	*Filters Count*
S	*Stride*

Problem: Image will vanish if applying more filters in cascade

- Our images get smaller and smaller
- Not too deep architectures
- Details are lost



Original 5x5

After apply one filter of size 3x3, output will be3x3

After applying second filter on the output (cascaded filters)

Output will be 1x1

Solution? Zero-padding! , Others !!

- For $s = 1$, surround the image with $(h_f - 1)/2$ and $(w_f - 1)/2$ layers of 0

0	0	0	0	0	0	0
0	1	1	1	0	0	0
0	0	1	1	1	0	0
0	0	0	1	1	1	0
0	0	0	1	1	0	0
0	0	1	1	0	0	0
0	0	0	0	0	0	0

$*$

0	0	1
0	1	1
1	1	1

$=$

1	1	2	0	0
0	1	1	1	0
0	0	1	2	1
1	0	2	1	0
0	1	1	3	0

Original 5x5

After padding ONE pixel (surrounding the image) it will be 7x7

After apply one filter of size 3x3 , output will be 5x5 again

Do the same for next applied filter

Apply Convolution as a NEW MODULE

Important

Since Convolution is a DOT product (between filter values and corresponding image values),
It can be applied as if it is a NN Layer
Input: image values, **Weights** : filter values, **Activation**: Linear, **Bias**:0

***Remember**: for training, it is required to have the gradient of Output w.r.t both Input and weights*

- θ_{ij} ⇒ Filter weights
- **r, c** ⇒ row, column
- a_{rc} ⇒ Activation value = Σ
- θ ⇒ Vevtor of all weights

- Activation function

$$a_{r,c} = \sum_{i=-a}^{a} \sum_{j=-b}^{b} x_{r-i,\,c-j} \cdot \theta_{ij}$$

- Essentially a dot product, similar to linear layer

$$a_{r,c} \sim x^{T}_{region} \cdot \theta$$

- Gradient w.r.t. the parameters

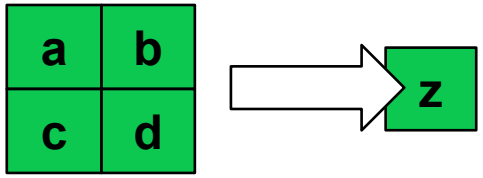
$$\frac{\partial a_{r,c}}{\partial \theta_{i,j}} = \sum_{r=0}^{N-2a} \sum_{c=0}^{N-2b} x_{r-i,\,c-j}$$

Weights: filter values

Input: pixel values

POOLING

- Aggregate multiple values into a single value
  - Invariance to small transformations
  - Reduces the size of the layer output/input to next layer → Faster computations
  - Keeps most important information for the next layer
- Max pooling
- Average pooling





Size is reduced

AVERAGE POOLING Layer (Take Average Value)

It is a new Module (Average-POOLING module)

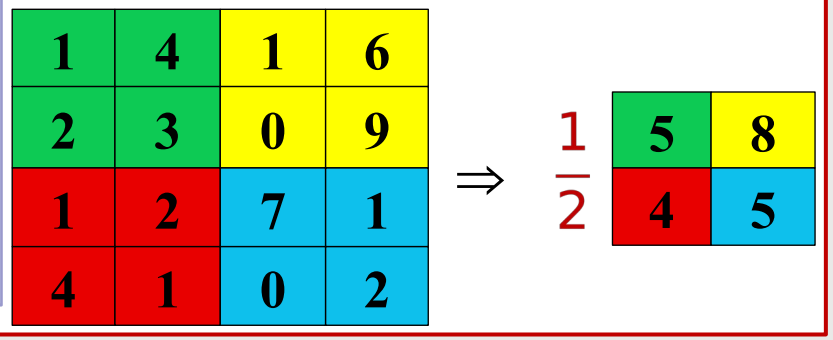
Output = 1/r c * Summation (each_input_pixel * 1)

Required to define its input, its output, and Gradient of Output w.r.t input

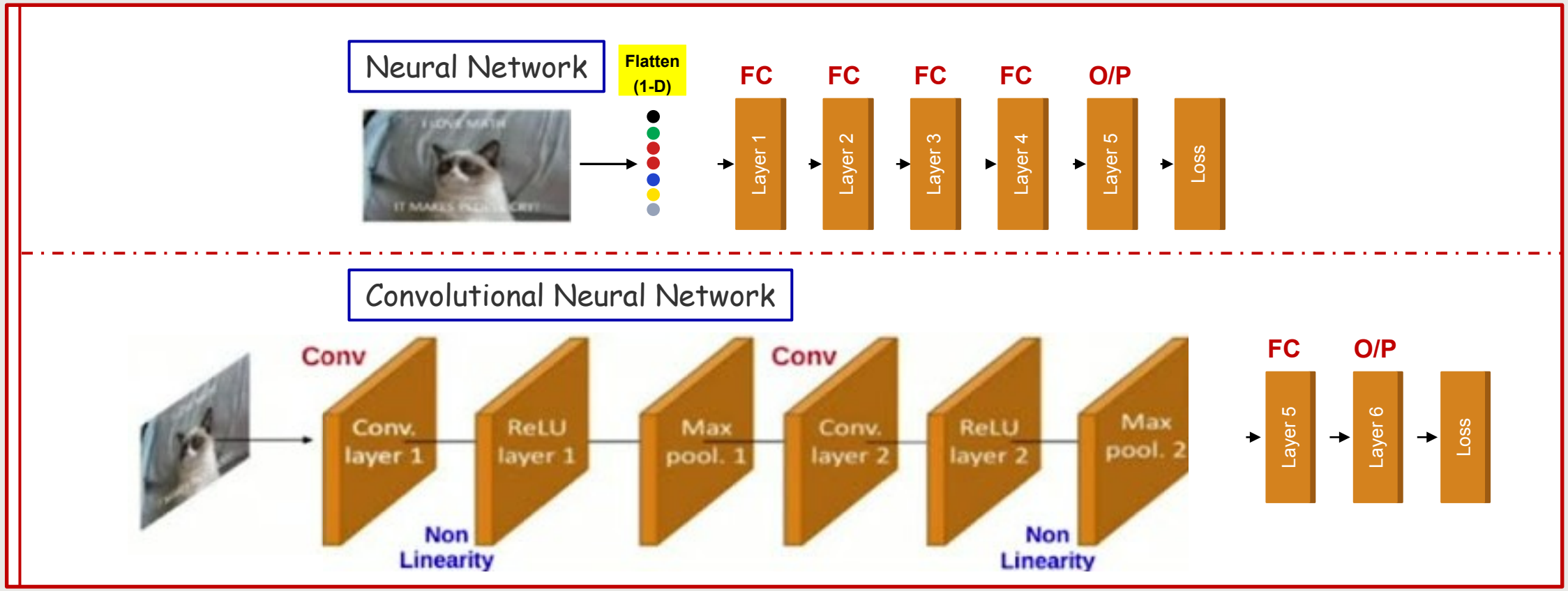
- Run a sliding window of size $[h_f , w_f]$
- At each location keep the **average** value
- Activation function: $a_{r,c} = \frac{1}{r \cdot c} \sum_{i,j \in \Omega(r,c)} x_{ij}$
- Gradient w.r.t. input $\frac{\partial a_{r,c}}{\partial x_{ij}} = \frac{1}{r \cdot c}$

Where:

r	*Row*
c	*Column*
f	*filter*
h	*Height*
w	*width*



Standard Neural Network (NN) vs Convolutional Neural Network (ConvNet/CNN)



Deep Learning (too Many Layers)

Convnets in practice

- Several convolutional layers (5 or more)
- After the convolutional layers **non-linearities** are added (The most popular one is the ReLU)
- After the ReLU usually some pooling (Most often max pooling)
- **After 5 rounds of cascading:**
  - Vectorize the last convolutional layer (Flatten)
  - Use as Input to the first fully connected layer (FC)
  - Proceed as in a traditional classification neural network